```
template<typename T>
class lock_free_queue
{
private:
    struct node
    {
        std::shared_ptr<T> data;
        node* next;

        node():
            next(nullptr)
        {}
    };

    std::atomic<node*> head;
    std::atomic<node*> tail;
    node* pop_head()

    {
        node* const old_head=head.load();
        if(old_head==tail.load())          <-- (1)
        {
            return nullptr;
        }
        head.store(old_head->next);
        return old_head;
    }
public:
    lock_free_queue():
        head(new node),tail(head.load())
    {}
```

At first glance, this doesn't seem too bad, and if there's only one thread calling `push()` at a time, and only one thread calling `pop()`, then this is actually perfectly fine. The important thing in that case is the happens-before relationship between the `push()` and the `pop()` to ensure that it's safe to retrieve the `data`. The store to `tail` ❼ synchronizes with the load from `tail` ❶; the store to the preceding node's `data` pointer ❺is sequenced before the store to `tail;` and the load from `tail` is sequenced before the load from the `data` pointer ❷, so the store to `data` happens before the load, and everything is OK. This is therefore a perfectly serviceable *single-producer, single-consumer (SPSC)* queue.